06

SKT K-뉴딜 아카데미 · AI· 자동화 기초

웹훅 · 터미널 CLI 인자

Flask로 요청 경로와 처리 함수를 연결하고 curl 로 요청을 보내 봅니다.
설정은 argparse 로 코드 밖에 뺍니다.

1교시 — 스스로 찾아서 공부합니다

아침 과제 문서를 열고, VS Code 에서 오늘 수업을 실행할 준비를 합니다.

모닝 30 과제

- `1002_fri/1002_morning30.md`
- 확장 설치부터 확인 셀 실행까지

오늘 할 일

- 확장 두 개 설치 — **Python · Jupyter**
- 터미널을 **Git Bash** 로 변경
- 패키지 네 개 설치
- 노트북 확인 셀 실행 → 준비 완료

> 확인 셀에서 준비 완료가 출력되어야 2교시를 시작할 수 있습니다.

2교시에 사용하는 용어는 여섯 개입니다

용어 (읽는 법)	한 줄 뜻
alert (얼러트 · 경보)	공격으로 의심되는 일을 찾았다는 알림
폴링 (polling)	정해진 간격으로 **API**를 조회하는 방식
웹훅 (webhook)	**alert** 가 발생하면 서버로 **HTTP** 요청을 보내 알리는 방식
서버 (server)	클라이언트의 요청을 받아 응답하는 프로그램
Flask (플라스크)	파이썬으로 서버를 만들 때 쓰는 도구 (웹 프레임워크)
라우트 (route)	어떤 요청에 어떤 함수를 실행할지 정한 규칙

클라이언트는 요청을 보내고, 서버는 요청을 받아 응답합니다

인터넷 서비스는 대부분 클라이언트와 서버, 두 역할로 나뉩니다.
두 역할로 나뉜 구조를 클라이언트-서버라고 합니다.

클라이언트 — 요청을 보내는 프로그램

- 필요할 때만 실행한다
- 예: 웹 브라우저 · 카카오톡 앱
- 주소창에 naver.com 을 입력하면 웹 브라우저가 요청을 보낸다

서버 — 요청을 받아 응답하는 프로그램

- 요청을 기다리며 계속 실행된다
- 예: 네이버 웹 서버 · 카카오 서버
- 요청을 받은 네이버 웹 서버가 첫 화면을 응답한다

신규 alert(경보)를 확인하는 방법은 폴링과 웹훅 두 가지입니다

SIEM(심 · 보안 정보·이벤트 관리 시스템): 로그를 모아 공격 시도를 탐지하고 alert 를 만드는 시스템.
신규 alert 가 생겼는지를 사람 대신 프로그램이 확인해야 합니다.
- 폴링: 정해진 간격으로 API를 조회하는 방식

폴링 — 내 프로그램이 5분마다 물어봅니다

폴링 스크립트

↓ 「신규 alert 가 있습니까?」

SIEM — 「있음」 또는 「없음」으로 응답

alert 를 최대 5분 뒤에 압니다

웹훅 — SIEM 이 먼저 알려 줍니다

SIEM

↓ alert 가 생기면 바로 보냄

웹훅 수신 서버 (오늘 만듭니다)

alert 를 생긴 즉시 압니다

5분마다 폴링하면 하루 288회 요청합니다

① **24시간 × 시간당 12회 = 288회**
신규 alert(경보)가 없어도 정해진 간격마다 요청을 보냅니다

② **alert 가 하루 3건이면 285회의 응답에는 신규 alert 가 없습니다**
288회 가운데 3회만 신규 alert 를 받습니다

③ **간격을 길게 하면 늦게 알고, 짧게 하면 요청이 늘어납니다**
5분 간격이면 alert 를 최대 5분 늦게 확인합니다. 1분 간격이면 하루 1,440회 요청합니다

웹훅은 alert 가 발생했을 때만 요청이 옵니다. alert 가 3건이면 요청도 3회입니다.

요청 URL은 호스트·포트·경로로 구성됩니다

```
http://127.0.0.1:5001/webhook
```

127.0.0.1 호스트	내 컴퓨터를 가리키는 주소
5001 포트 번호	포트: 한 컴퓨터에서 실행 중인 서버 프로그램을 구분하는 번호. 서버마다 다른 번호를 사용
/webhook 경로	서버 안에서 어떤 함수를 실행할지 정하는 이름

서버는 요청마다 다른 함수를 실행해서 연결합니다. (라우트)

함수를 미리 만들어 두고 「이 요청에는 이 함수」라고 연결해 둡니다. 이 연결이 라우트입니다. 요청이 오면 서버가 그 함수를 실행합니다.

요청	실행할 함수	하는 일
`POST /webhook`	`webhook()`	alert(경보)를 받아 저장한다 — 오늘 만듭니다
`GET /health`	`health()`	서버가 실행 중인지 알려 준다 (예시)
`GET /alerts`	`alerts()`	받은 alert 목록을 돌려준다 (예시)

라우트가 없으면 요청이 와도 실행할 함수가 없습니다. 서버는 `404` 를 반환합니다.

요청이 오면 Flask(파이썬으로 웹서버 만드는 도구) 가 순서대로 함수를 찾아 실행합니다

요청 도착
POST /webhook
→
함수 찾기
webhook()
→
함수 실행
return
→
응답
{"status":"ok"}

① **클라이언트가 /webhook 으로 POST 요청을 보냅니다**
요청이 서버에 도착합니다

② **Flask 가 /webhook 에 연결된 함수 webhook() 을 찾아 실행합니다**
「이 요청에는 이 함수」라는 연결을 **라우트(route)**라고 합니다

③ **함수가 return 한 값을 클라이언트에 응답합니다**
클라이언트는 {"status":"ok"} 를 받습니다

객체는 점 . 뒤에 쓸 수 있는 기능을 가진 값입니다

```
logs = ["로그1", "로그2"]      # logs 는 리스트입니다
logs.append("로그3")            # 점 뒤의 append() 는 리스트가 가진 기능입니다
```

① **리스트는 객체입니다**

logs 는 리스트이고, 리스트는 append() 라는 기능을 가지고 있습니다.
파이썬에서는 리스트 · 문자열 · 딕셔너리가 모두 객체입니다

② **request 도 객체입니다**

request.get_json() — 점 뒤의 get_json() 은 request 가 가진 기능입니다.
요청 본문을 딕셔너리로 변환합니다

쓰는 방법이 같습니다. 객체 이름 · 점 · 기능 이름 · 괄호 순서로 적습니다.

@ 줄은 아래 함수를 실행하라는 표시입니다

웹훅(webhook): 일이 생기면 상대 프로그램이 내 서버로 먼저 요청을 보내 알려 주는 방식입니다.

「/webhook 으로 POST 요청이 오면」: 누군가 내 서버의 /webhook 주소로 데이터를 보내 오면이라는 뜻입니다.

```
@app.route("/webhook", methods=["POST"])
def webhook():
```

@ 는 「앳」이라고 읽고, 함수 바로 윗줄에 씁니다.
@ 줄이 없으면 요청이 와도 webhook() 은 실행되지 않습니다.

Flask 로 요청을 받을 라우트를 만듭니다

라우트는 요청과 처리 함수를 연결하는 규칙입니다.
`request` 는 **Flask** 가 받은 요청을 담은 객체입니다. 앞에서 배운 `requests` 라이브러리와 다릅니다.

```
from flask import Flask, request

app = Flask(__name__)                          # 서버를 만들어 app 에 저장한다

@app.route("/webhook", methods=["POST"])    # /webhook 경로에 POST 요청이 오면
def webhook():                                 # webhook() 함수가 실행된다
    event = request.get_json()                  # JSON 본문을 파이썬 값으로 읽는다
    return {"status": "ok"}, 200                # 응답 본문과 상태 코드를 반환한다
```

요청 본문은 문자열이라서 딕셔너리로 변환합니다

① **클라이언트가 보낸 요청 본문은 JSON 문자열입니다**
`'{"rule": "brute_force", "user": "admin"}'`

② **문자열에서는 값을 꺼낼 수 없습니다**
문자열에 `["rule"]` 을 쓰면 오류가 발생합니다

③ **`event = request.get_json()` 으로 딕셔너리로 변환합니다**
변환한 뒤에는 `event["rule"]` 로 `'brute_force'` 를 꺼낼 수 있습니다

> 응답은 반대입니다.
> 함수가 딕셔너리 `{"status": "ok"}` 를 `return` 하면 **Flask 가 JSON 문자열로 변환해 보냅니다.**

서버 프로그램은 요청을 기다리며 계속 실행됩니다

① **실행 상태를 유지하는 서버 프로세스**
`app.run()` 은 요청을 기다립니다. 요청이 언제 올지 모르므로 종료하지 않습니다

② **노트북 셀에서 실행하면 그 셀이 끝나지 않습니다**
셀이 끝나지 않으면 다음 셀을 실행할 수 없습니다. 오류가 아닙니다

③ **그래서 서버를 백그라운드에서 실행합니다**
백그라운드에서 실행하면 서버는 뒤에서 계속 실행되고, 우리는 다음 셀을 실행할 수 있습니다

포그라운드와 백그라운드는 기다리는지가 다릅니다

포그라운드(foreground): 프로그램이 끝날 때까지 기다리는 실행 방식입니다.
백그라운드(background): 프로그램을 뒤에서 실행해 두고 바로 다음 일을 하는 실행 방식입니다.

포그라운드 — 끝날 때까지 기다립니다

셀에서 `app.run()` 을 실행

↓ 서버는 끝나지 않음

셀이 끝나지 않음

다음 셀을 실행할 수 없습니다

백그라운드 — 뒤에서 실행해 둡니다

셀에서 `start_server()` 를 실행

↓ 서버는 뒤에서 계속 실행

셀이 바로 끝남

다음 셀을 실행할 수 있습니다

서버는 끝나지 않는 프로그램이므로 백그라운드에서 실행합니다.

start_server() 로 서버를 백그라운드에서 실행합니다

서버 코드를 hello_server.py 파일로 저장한 뒤 실행합니다.
start_server() 는 노트북 맨 위 셀에 정의되어 있습니다.

```
# hello_server.py 의 마지막 줄 - 포트 5001 에서 요청을 기다린다
app.run(port=5001)


# 노트북 셀 - hello_server.py 를 백그라운드에서 실행한다
server1 = start_server("hello_server.py")
```

```
hello_server.py 를 실행했습니다
```

서버가 받은 alert(경보)를 파일에 저장합니다

① **`received` 리스트에 alert 데이터 추가**
리스트는 서버 프로그램의 변수입니다. 서버를 종료하면 리스트의 값은 사라집니다

② **그래서 요청을 받을 때마다 파일에 저장합니다**
앞에서 배운 `json.dump()` 로 결과를 파일에 쓰는 대신 **JSON 문자열로 저장**합니다

③ **응답에 받은 alert 의 수를 넣습니다**
두 번째 요청의 응답은 `{"count":2,"status":"ok"}` 입니다

webhook() 함수에 파일 저장 코드를 추가합니다

```
import json
from flask import Flask, request
app = Flask(__name__)
received = []
@app.route("/webhook", methods=["POST"])
def webhook():
    received.append(request.get_json())
    with open("received_alerts.json", "w", encoding="utf-8") as f:
        json.dump(received, f, ensure_ascii=False, indent=2)
    return {"status": "ok", "count": len(received)}, 200
app.run(port=5006)   # 5001 은 hello_server.py 가 사용 중이다
```

상태 코드는 요청 처리 결과를 나타내는 세 자리 숫자입니다

상태 코드	이름	뜻
404	Not Found	요청한 경로가 서버에 없다
405	Method Not Allowed	경로는 있지만 요청한 HTTP 메서드를 허용하지 않는다 (보낸 요청 방식은 그 주소에서 받지 않음)

> `/webhook` 라우트는 `methods=["POST"]` 로 만들었습니다.
> **GET 요청은 허용하지 않으므로 서버가 405 를 반환합니다.**

실습 ① · 노트북 2교시 절 · 261002_am_webhook_cli.ipynb

문제 2-1 ~ 2-9 · 도전 ⭐2-5 · ⭐2-10

① **맨 위 start_server() 함수 정의 셀을 먼저 실행합니다**

실행하지 않으면 뒤 셀에서 NameError 가 발생합니다

② **문제 2-1~2-4 — 조회 결과와 반복 횟수**

2-1 은 먼저 답을 적습니다. 결과가 없는 조회 횟수를 셉니다

③ **문제 2-6 ~ 2-9 — 서버를 실행한 뒤 요청을 보낸다**

💻 표시가 있는 curl 명령은 터미널에 그대로 입력합니다. curl 은 3교시에 배웁니다

2교시에 배운 것을 다시 확인합니다

배운 것	뜻
폴링 (polling)	정해진 간격으로 조회한다. 간격을 줄이면 요청 횟수가 늘어난다
웹훅 (webhook)	**alert(경보)**가 발생하면 **SIEM** 이 요청을 보낸다. 웹훅 수신 서버가 필요하다
`@app.route("/webhook", methods=["POST"])`	**POST** 요청이 오면 아래 함수를 실행한다
`request.get_json()` · `return {...}, 200`	요청 본문을 딕셔너리로 변환하고, 응답 본문과 상태 코드를 반환한다

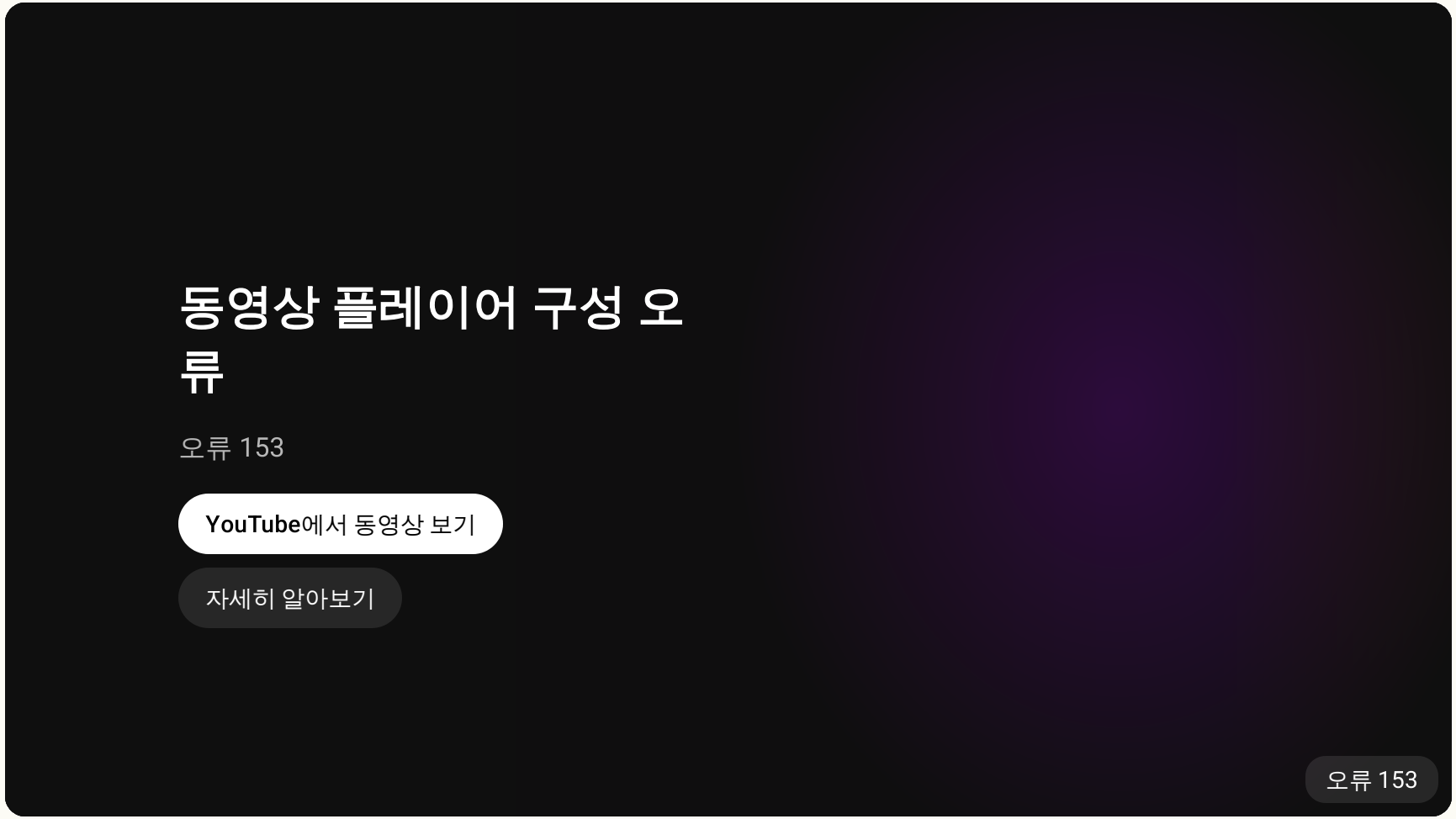
3교시에 사용하는 용어는 다섯 개입니다

용어 (읽는 법)	한 줄 뜻
CLI (씨엘아이 · 명령줄 인터페이스)	명령을 글자로 입력해 프로그램을 사용하는 방식
GUI (지유아이 · 그래픽 사용자 인터페이스)	창·메뉴·버튼을 눌러 프로그램을 사용하는 방식
셸 (shell)	입력한 명령을 해석해 실행하는 프로그램. 예: bash(배시)
curl (컬)	HTTP 요청을 전송하는 명령줄 도구
-X · -H · -d (대시 엑스 · 대시 에이치 · 대시 디)	HTTP 메서드 · 요청 헤더 · 요청 본문을 지정하는 옵션

CLI 명령은 스크립트로 저장해 반복 실행할 수 있습니다

비교 기준	GUI	CLI
조작 방법	창·메뉴·버튼을 마우스로 조작한다	터미널에 명령을 입력한다
같은 작업 반복	화면 조작을 처음부터 다시 한다	명령을 셸 스크립트에 저장해 재실행
작업 공유	화면 캡처나 절차 문서를 만든다	명령 텍스트를 그대로 전달한다
실행 환경	그래픽 데스크톱 환경이 필요하다	그래픽 환경이 없는 서버에서도 실행한다

5분 영상으로 GUI 와 CLI 의 차이를 한 번 더 봅니다



리눅스 명령어 cd · ls · cat · grep

명령어 (읽는 법)	하는 일	입력 예 → 결과
`cd` (씨디)	작업 폴더를 이동한다	`cd agent_core` → **agent_core** 폴더로 이동
`ls` (엘에스)	작업 폴더의 파일 목록을 출력한다	`ls` → 파일 이름 목록 출력
`cat` (캣)	파일 내용을 출력한다	`cat hello_server.py` → 파일 내용 전체 출력
`grep` (그렙)	지정한 문자열이 있는 줄만 출력한다	`grep port hello_server.py` → **port** 가 있는 줄만 출력

서버는 POST 요청을 받으면 응답을 돌려줍니다

POST 요청: 클라이언트가 서버에 데이터를 보내는 요청입니다.
응답(response): 요청을 받은 서버가 돌려주는 답입니다.

① 요청
클라이언트가 alert 를 담아 POST 요청을 보냄

→

② 처리
서버가 webhook() 을 실행

→

③ 응답
200 과 {"status": "ok"} 를 돌려줌

응답에 들어 있는 것	값	뜻
상태 코드	`200`	정상 처리. 보낸 쪽은 이 숫자로 전달됐다는 것을 압니다
본문	`{"status": "ok"}`	서버가 덧붙여 보내는 내용. 우리가 쓴 `return` 줄입니다

curl 은 HTTP 요청을 전송하고 응답 본문을 출력합니다

```
curl -s -X POST http://127.0.0.1:5001/webhook
```

`curl`	서버에 요청을 보내는 도구. 받은 응답을 화면에 출력
`-s`	**silent**(조용히)의 약자. 진행 상황과 오류 메시지를 화면에 표시하지 않음
`-X POST`	긴 이름은 **--request**(요청). 데이터를 보내는 방식으로 요청. 적지 않으면 **GET**
URL	요청을 보낼 주소

-H 는 요청 헤더를, -d 는 요청 본문을 지정합니다

```
curl -s -X POST -H "Content-Type: application/json" \
  -d '{"rule":"night_login"}' http://127.0.0.1:5001/webhook
```

```
{"status":"ok"}
```

`curl`	서버에 요청을 보내는 도구
`-s`	진행 상황을 화면에 표시하지 않음
`-X POST`	데이터를 보내는 방식으로 요청
`-H "Content-Type: application/json"`	**header**(헤더)의 약자. 보내는 내용이 **JSON** 임을 알림
`\`	명령이 다음 줄에 이어진다는 표시
`-d '{"rule":"night_login"}'`	**data**(데이터)의 약자. `rule` 에 `night_login` 을 담아 보냄

curl 로 /webhook 에 POST 요청을 보내면 {"status":"ok"} 가 반환됩니다

줄 끝의 \(역슬래시)는 명령이 다음 줄에 이어진다는 표시입니다.

아래 명령은 노트북 셀이 아니라 터미널에 입력합니다.
2교시에 실행한 서버에 curl(컬) 명령으로 요청을 보냅니다.

```
curl -s -X POST -H "Content-Type: application/json" \
  -d '{"rule":"brute_force"}' http://127.0.0.1:5001/webhook
```

```
{"status":"ok"}
```

밑줄은 의미 단위입니다 — 명령 · HTTP 메서드 · 요청 헤더 · 요청 본문 · URL

-o 와 -w 를 지정하면 상태 코드만 출력됩니다

```
curl -s -o /dev/null -w "%{http_code}" \
  http://127.0.0.1:5001/webhook
```

```
405
```

`-o /dev/null`	**output**(출력)의 약자. 응답 본문을 화면에 표시하지 않음. `/dev/null`(데브 널)은 받은 내용을 버리는 곳
`-w "%{http_code}"`	**write-out**(써서 내보내기)의 약자. 상태 코드를 화면에 표시
405	`-X POST` 를 적지 않아 **GET** 요청으로 보낸 결과

JSON 전체는 작은따옴표로 감싸고, JSON 안에서는 큰따옴표를 씁니다

```
curl -s -X POST -H "Content-Type: application/json" \
  -d '{"rule":"night_login"}' http://127.0.0.1:5001/webhook
```

작은따옴표 '…'	JSON 전체를 감쌈. 큰따옴표로 감싸면 안의 큰따옴표와 섞여 명령이 중간에 끊김
큰따옴표 "rule"	JSON 안의 이름과 값을 감쌈. JSON 은 큰따옴표만 허용
\	뒤에 공백이 있으면 다음 줄로 이어지지 않음

오류를 나타내는 HTTP 상태 코드는 세 가지를 사용합니다

상태 코드	뜻	발생하는 경우
404	요청한 경로가 없다	경로를 잘못 지정했을 때
405	경로는 있지만 요청 메서드가 허용되지 않는다	GET 요청을 보냈을 때
415	서버가 요청 본문 형식을 처리할 수 없다	-H 옵션을 생략했을 때

> 415 는 오늘 처음 나옵니다. 200 번대는 정상 처리, 400 번대는 클라이언트 요청에 문제가 있다는 뜻입니다.

`-H` 옵션을 생략하면 서버가 415 를 반환합니다

```
curl -s -o /dev/null -w "%{http_code}" -X POST \
  -d '{"rule":"night_login"}' http://127.0.0.1:5001/webhook
```

```
415
```

`-H` 없음	「보내는 내용은 **JSON** 형식입니다」라고 서버에 알리지 않음
415	서버가 「보낸 내용의 형식을 처리할 수 없습니다」라고 응답

셸 스크립트에 명령을 저장해 반복 실행합니다

명령을 순서대로 적어 둔 파일을 셸 스크립트라고 합니다. 파일 이름은 .sh 로 끝납니다.

저장
명령을 파일에 저장

→

실행
bash demo_test.sh

→

실행 결과
저장한 순서대로 실행

① **명령을 파일에 순서대로 저장합니다**
예: demo_test.sh 파일에 curl 명령 두 개를 적습니다

② **bash demo_test.sh 로 실행합니다**
bash(배시)는 셸 프로그램입니다. 파일에 적힌 명령을 위에서부터 차례로 실행합니다

③ **다시 실행하면 같은 명령이 같은 순서로 실행됩니다**
명령을 다시 입력하지 않으므로 입력 오타가 생기지 않습니다

반복할 curl 명령을 demo_test.sh 파일에 저장합니다

```
save_script("demo_test.sh", r"""
curl -s -X POST -H "Content-Type: application/json" \
  -d '{"rule":"brute_force"}' http://127.0.0.1:5001/webhook
echo
curl -s -X POST -H "Content-Type: application/json" \
  -d '{"rule":"night_login"}' http://127.0.0.1:5001/webhook
echo
""")
```

save_script("demo_test.sh", r"""…""")	따옴표 세 개 사이의 내용을 demo_test.sh 파일로 저장 (노트북 셀에서 실행)
echo (에코)	빈 줄을 출력. 응답 두 개가 붙어 보이지 않게 함

`bash demo_test.sh` 를 실행하면 curl 명령 두 개가 차례로 실행됩니다

```
bash demo_test.sh
```

```
{"status":"ok"}


{"status":"ok"}
```

`bash` (배시)	파일에 적힌 명령을 위에서부터 차례로 실행
`demo_test.sh`	실행할 셸 스크립트 파일
응답 사이의 빈 줄	`echo` 가 출력

3교시에 배운 것을 정리합니다

쓰는 법	뜻
`curl -X POST`	HTTP 메서드를 지정한다
`-H "Content-Type: application/json"`	요청 헤더로 요청 본문 형식을 지정한다. 생략하면 **415**
`-d '{...}'`	요청 본문을 지정한다
`-o /dev/null -w "%{http_code}"`	응답 본문은 출력하지 않고 상태 코드만 출력한다
`bash demo_test.sh`	curl 명령을 저장한 파일을 한 번의 명령으로 실행한다

실습 ② · 노트북 3교시 절

문제 3-1 ~ 3-9 · 도전 ⭐3-5 · ⭐3-10

① **2교시에 실행한 포트 5001의 서버를 계속 사용합니다**

종료했다면 노트북의 `start_server()` 함수로 다시 실행합니다

② **문제 3-1 ~ 3-4 — `curl` 로 요청 보내기**

명령은 터미널에 입력합니다. 3-1 은 실행 전에 결과를 먼저 적습니다

③ **문제 3-6 ~ 3-9 — `test_webhook.sh`**

오전 산출물 `test_webhook.sh` 를 만듭니다

-H 는 필요할 때만 쓰고, 옵션 순서는 바꿔도 됩니다

```
curl -s -X POST -H "Content-Type: application/json" \
  -d '{"rule":"night_login"}' http://127.0.0.1:5001/webhook
```

질문	답
-H 는 항상 써야 합니까?	아닙니다. 요청에 헤더를 추가할 때만 씁니다. 오늘은 JSON 을 보내므로 「보내는 내용은 JSON 입니다」라고 알리려고 넣습니다.
옵션 순서를 바꿔도 됩니까?	됩니다. -s · -X POST · -H · -d 는 순서를 바꿔도 결과가 같습니다.
URL 은 어디에 둡니까?	맨 뒤에 둡니다. 요청을 보낼 대상이므로 눈에 잘 띄게 합니다.

> 지금은 수업에서 본 순서 그대로 입력합니다.

노트북이 이상하면 맨 위에서 **Restart** 한 뒤 **Run All** 을 누릅니다

노트북은 실행한 셀의 변수와 함수를 기억합니다.
셀을 고치거나 지워도 그 기억은 그대로 남습니다.

이런 경우	누를 것	이유
위쪽 셀을 건너뛰어 `NameError` 가 납니다	**Run All**	실행하지 않은 셀의 변수는 기억에 없습니다
셀을 고쳤는데 예전 값이 나옵니다	**Restart → Run All**	예전 값이 기억에 남아 있습니다. 비우고 다시 실행합니다
셀이 끝나지 않고 계속 실행됩니다	**Restart**	실행을 강제로 끝냅니다

Restart: 기억을 모두 지웁니다. 셀에 적은 코드는 그대로 남습니다.
Run All: 맨 위 셀부터 차례로 실행해 기억을 다시 채웁니다.

4교시에 사용하는 용어는 네 개입니다

용어 (읽는 법)	한 줄 뜻
CLI 인자 (씨엘아이 인자)	실행 명령에서 프로그램에 전달하는 값
argparse (아그파스)	실행 명령의 인자를 읽는 파이썬 기본 모듈. 설치하지 않아도 된다
--port (대시 대시 포트)	이름 뒤에 값을 적는 인자. 옵션이라고도 한다. **curl** 의 `-X POST` 와 같은 형식이다
기본값	인자를 생략했을 때 사용하는 값

포트 번호를 코드에 쓰지 않고 실행 명령의 인자로 지정합니다

포트 5001 을 다른 서버가 사용 중이면 포트 번호를 바꿔야 합니다.
실행 명령에서 지정하면 코드 수정 없이 포트 변경

변경 전 — 포트 번호가 코드 안에 있다

```
app.run(port=5001)
```

포트를 바꾸려면 파일을 고쳐야 한다

변경 후 — 실행할 때 인자로 지정한다

```
python webhook_server.py --port 5001
```

파일은 그대로 두고 값만 바꾼다

인자는 실행 명령에서 프로그램에 전달하는 값입니다 (예: --port 5001)

```
python webhook_server.py --port 5001
```

`python`	파이썬으로 파일을 실행
`webhook_server.py`	실행할 파일. 서버 코드가 들어 있음
`--port 5001`	파일에 전달하는 인자. `--port` 가 이름이고 `5001` 이 값

포트 번호를 인자로 전달할 때의 장점
코드를 고치지 않고 실행할 때 포트를 바꿀 수 있습니다.
기본 포트 5001 을 다른 프로그램이 사용 중이면 `--port 5002` 처럼 다른 포트로 실행합니다.

argparse 는 세 단계로 인자를 읽습니다

```
parser = argparse.ArgumentParser()
parser.add_argument("--port", type=int, default=5000)
args = parser.parse_args()
```

① ArgumentParser()	인자를 읽을 준비를 함
② add_argument("--port", …)	--port 라는 인자를 받겠다고 등록
type=int	받은 값을 정수로 바꿈
default=5000	--port 를 적지 않으면 **5000** 을 사용
③ parse_args()	실행 명령에 적힌 인자를 읽음
args.port	읽은 포트 번호. --port 5001 로 실행하면 **5001**

argparse 로 --port 인자를 읽습니다

앞 장의 세 단계를 코드로 작성했습니다. port_demo.py 파일로 저장한 뒤 터미널에서 실행합니다. help 에 작성한 설명은 --help 를 지정했을 때 출력됩니다.

```
import argparse

parser = argparse.ArgumentParser(description="포트를 받아 출력해 봅니다")
parser.add_argument("--port", type=int, default=5000,
                    help="열어 둘 포트 번호")
args = parser.parse_args()

print("받은 포트:", args.port)
```

`--port` 를 생략하면 기본값 5000 을,
지정하면 지정한 값을 사용합니다

> 앞 장의 `port_demo.py` 를 실행했습니다. 파일은 그대로이고 인자 값만 바꿨습니다.

```
python port_demo.py
```

```
받은 포트: 5000
```

```
python port_demo.py --port 5002
```

```
받은 포트: 5002
```

`--help` 를 지정하면 help 에 작성한 설명이 출력됩니다

description 과 help 에 작성한 설명이 출력됩니다. -h, --help 는 **argparse** 가 자동으로 추가합니다.

```
python port_demo.py --help
```

```
usage: port_demo.py [-h] [--port PORT]

포트를 받아 출력해 봅니다

options:
  -h, --help   show this help message and exit
  --port PORT  열어 둘 포트 번호
```

`type=int` 를 생략하면 값이 문자열로 들어옵니다

실행 명령에 입력한 값은 문자열입니다.
`type=int` 를 쓰면 정수로 변환됩니다.

두 경우 모두 `--port 5002` 로 실행했습니다.

type=int 생략 — args.port 는 문자열 "5002"

- `args.port + 1` 은 TypeError 가 발생합니다
- 문자열에 숫자를 더할 수 없습니다

type=int 지정 — args.port 는 정수 5002

- `args.port + 1` 의 결과는 5003 입니다
- 포트 번호는 정수로 받습니다

오전에 만든 기능을 한 파일에 모읍니다

기능	배운 교시	하는 일
Flask 라우트	2교시	/webhook 경로의 POST 요청을 처리한다
수신한 alert(경보) 데이터 저장	2교시 문제 2-9	received_alerts.json에 저장한다
argparse --port	4교시	app.run(port=args.port) 로 서버를 실행한다

새 문법은 없습니다. 오전에 만든 기능을 서버 파일 하나에 모읍니다.
파일 이름은 학원 교안에 지정된 webhook_server.py 입니다.

4교시에 배운 것을 정리합니다

쓰는 법	뜻
`parser.add_argument("--port", type=int, default=5000)`	`--port` 인자를 등록한다. 자료형은 정수, 기본값은 **5000**
`args.port`	읽은 인자 값을 사용한다
`--help`	`help` 에 작성한 설명을 출력한다
`type=int`	생략하면 실행 명령에 입력한 값이 문자열로 들어온다

> 오전 산출물은 `webhook_server.py` 와 `test_webhook.sh` 입니다.

부트텐트에서 KDT 과정을 한눈에 비교합니다

이름	무엇인가
KDT K-디지털 트레이닝	고용노동부가 훈련비를 지원하는 디지털 분야 직업 훈련 과정입니다. 국민내일배움카드로 수강합니다.
부트텐트 boottent.com	부트캠프와 KDT 과정을 모아 한눈에 비교하는 사이트입니다. 분야 · 무료/유료 · 온라인/오프라인으로 찾습니다.

👉 https://boottent.com/

실습 ③ · 노트북 4교시 절

문제 4-1 ~ 4-9 · 도전 ⭐4-5 · ⭐4-10

① **문제 4-1 ~ 4-4 — argparse**

· 4-4 는 type=int 를 생략했을 때의 결과를 확인합니다

② **문제 4-7 — webhook_server.py**

오전의 최종 실습. 문제 2-9 에서 만든 save_server.py 에 --port 인자를 추가합니다

③ **문제 4-8 · 4-9 — 서버 실행과 agent_core 폴더 저장**

노트북 명령 %cd agent_core 로 작업 폴더를 이동한 뒤 webhook_server.py 와 test_webhook.sh 를 만드는 셀을 다시 실행합니다